

Data Structures Notes: Trees

1. Introduction to Trees

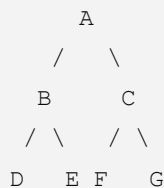
A **Tree** is a non-linear data structure used to represent hierarchical relationships between elements.

Unlike arrays, stacks, queues, and linked lists, where data is generally organized sequentially, a tree organizes data in **levels**.

Real-life examples

- File and folder systems
- Organization hierarchy
- HTML/XML document structure
- Database indexing
- Expression evaluation
- Artificial intelligence decision trees
- Network routing

Basic Tree Example

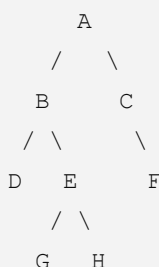


Here:

- **A** is the root.
- **B** and **C** are children of **A**.
- **D, E, F, G** are leaf nodes.

2. Basic Tree Terminologies

Consider the following tree:



2.1 Node

A **node** is an individual element of a tree.

A

Here, A is a node.

A node generally contains:

```
+-----+-----+
|  Data  |  Link  |
+-----+-----+
```

2.2 Edge

An **edge** is the connection between two nodes.

A
|
B

The connection between A and B is an edge.

If a tree contains n nodes, it always contains:

$n - 1$ edges

2.3 Root

The **root** is the topmost node of a tree.

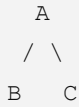
```
    A      ← Root
   /  \
  B    C
```

Here, A is the root.

A tree has **exactly one root node**.

2.4 Parent

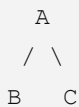
A node that has one or more children is called a **parent**.



A is the parent of B and C.

2.5 Child

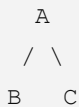
A node directly connected below another node is called its **child**.



B and C are children of A.

2.6 Siblings

Nodes having the same parent are called **siblings**.



B and C are siblings.

In the larger tree:



D and E are siblings.

2.7 Leaf Node

A node having **no children** is called a **leaf node**, terminal node, or external node.



D and **C** are leaf nodes.

2.8 Internal Node

A node having at least one child is called an **internal node** or non-leaf node.



A and **B** are internal nodes.

2.9 Degree of a Node

The **degree of a node** is the number of children it has.



Degree of **A** = **3**

Degree of **B** = **0**

2.10 Degree of a Tree

The **degree of a tree** is the **maximum degree of any node** in the tree.



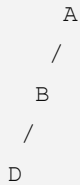
Maximum number of children of a node = 3.

Therefore:

Degree of tree = 3

2.11 Path

A **path** is a sequence of nodes connected by edges.



Path from **A** to **D**:

A → B → D

Number of edges in this path = 2.

2.12 Path Length

The number of edges in a path is called its **path length**.

A → B → D → G

Path length = 3

2.13 Level of a Node

The level represents the **position of a node relative to the root.**

One common convention is:

Level 0:	A
Level 1:	B C
Level 2:	D E F G

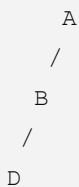
Thus:

- $A \rightarrow \text{Level } 0$
- $B, C \rightarrow \text{Level } 1$
- $D, E, F, G \rightarrow \text{Level } 2$

Note: Some textbooks define the root as Level 1. Always follow the convention specified in your course.

2.14 Depth

The **depth** of a node is the number of edges from the root to that node.



Depth:

- $A = 0$
- $B = 1$
- $D = 2$

2.15 Height of a Node

The height of a node is the **number of edges on the longest downward path** from that node to a leaf.

For example:



Height of:

- $D = 0$
- $E = 0$
- $B = 1$
- $C = 0$

- $A = 2$
-

2.16 Height of a Tree

The height of a tree is the height of its root.



Longest path:

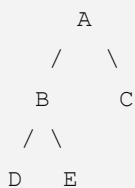
$A \rightarrow B \rightarrow C \rightarrow D$

Therefore:

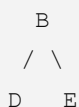
Height = 3 edges

2.17 Subtree

A **subtree** is a tree formed by a node and all of its descendants.

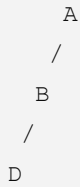


Subtree rooted at B :



2.18 Ancestor

A node is an **ancestor** of another node if it occurs on the path from the root to that node.

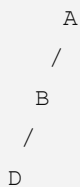


Ancestors of **D** :

B, A

2.19 Descendant

A node below another node is its descendant.



Descendants of **A** :

B, D

3. Types of Trees

There are many types of trees:

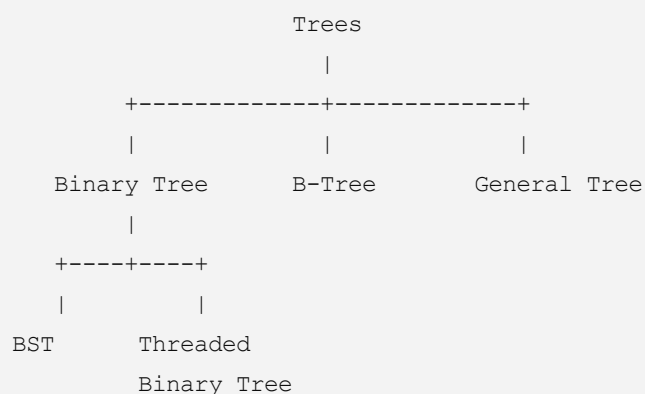
Different types of Trees:

Binary Tree
Binary Search

Tree

AVL Tree

Threaded Binary Tree



In this section, we focus on:

1. Binary Tree
2. Threaded Binary Tree